

Advanced Employee Management System

Using Python Tkinter & Oracle Database

Database Systems (DBS) _ Assignment 02

Session: 2K24

Submitted By: Khuram Sohail

Reg No: 24-SE-72

Instructor: Engr Saba Awan

Department of Software Engineering

1. Introduction

This report documents the development of an Advanced Employee Management System (EMS) built using Python Tkinter as the graphical user interface and Oracle Database as the backend data storage. The system is designed to manage employee records efficiently with full CRUD (Create, Read, Update, Delete) functionality, a secure role-based login system, smart search and filtering, salary statistics, data export, and real-time search features.

The project fulfills the requirements of DBS Assignment 02, addressing all three CLOs: practical application of database concepts, demonstration of original logical solutions, and teamwork/individual contribution.

2. Objective

The objective of this assignment is to enhance a previously developed Employee Management System by implementing:

- Secure login with role-based access control (ADMIN / USER)
- Full CRUD operations connected to Oracle Database
- Smart dynamic multi-field search and salary range filtering
- Input validation and duplicate prevention
- Statistics module showing salary analytics
- Export functionality to CSV and TXT formats
- Real-time live search while typing
- Multi-level sorting and conditional row highlighting

3. Technologies Used

Technology	Purpose
Python 3.x	Core programming language for application logic
Tkinter	GUI framework for building the desktop interface
Oracle Database	Backend relational database for storing employee and user data
oracledb (cx_Oracle)	Python library for Oracle Database connectivity
SQL	Structured Query Language for database operations
ttk.Treeview	Tkinter widget for displaying employee records in a table

4. Database Design

4.1 Employees Table

The main table storing all employee records:

Column	Data Type	Description
EmpID	NUMBER	Primary Key — unique employee identifier
Name	VARCHAR2(50)	Employee full name
Gender	VARCHAR2(10)	Male / Female
Department	VARCHAR2(50)	Department name (e.g., CS, Civil, SE)
Salary	NUMBER	Monthly salary amount
Contact	VARCHAR2(20)	Employee contact number

4.2 USERS Table

Stores user credentials and roles for the login system:

Column	Data Type	Description
Username	VARCHAR2(50)	Primary Key — login username
Password	VARCHAR2(50)	Login password (stored as text)
Role	VARCHAR2(10)	CHECK constraint: ADMIN or USER

4.3 Sample Data

The following sample records were inserted into the Employees table for testing:

EmpID	Name	Gender	Department	Salary	Contact
1	Arslan Fareed	Male	SE	1200	1232123
2	Khuram	Male	SE	1500	03313235733
3	Maryam	Female	Nursing	1000	0323423443
4	Asad	Male	Civil	1700	0323423221
5	Tayyaba	Female	CS	1600	03234232211

5. Implemented Features

5.1 Login System with Role-Based Access Control

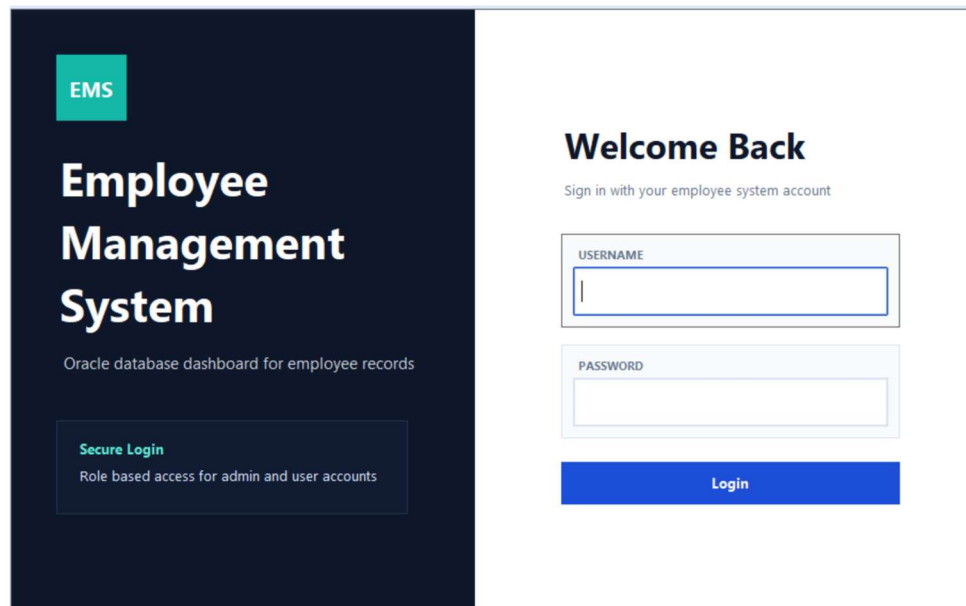
A dedicated Login Window serves as the entry point to the application. It contains a Username field, a password field (masked with asterisks), and a Login button. Credentials are verified against the USERS table in Oracle Database using parameterized SQL queries to prevent SQL injection. If credentials are incorrect, an error message is displayed.

Upon successful login:

- ADMIN role gains full access to all features (CRUD, search, export, statistics)
- USER role gets read-only access (view and search only; Insert, Update, Delete, and Export buttons are disabled and greyed out)

The logged-in username and role are displayed prominently in the main dashboard header (e.g., admin | ADMIN or user1 | USER).

Login Window:



The screenshot shows the login interface for the Employee Management System (EMS). On the left, a dark blue sidebar contains the EMS logo, the title 'Employee Management System', the subtitle 'Oracle database dashboard for employee records', and a 'Secure Login' section stating 'Role based access for admin and user accounts'. The main area is white and features a 'Welcome Back' message, a sign-in instruction, and two input fields for 'USERNAME' and 'PASSWORD'. A blue 'Login' button is positioned below the password field.

Admin Dashboard



The screenshot displays the Admin Dashboard for the Employee Management System. The browser window title is 'Advanced Employee Management System'. The dashboard has a blue header with the system name and the user 'admin | ADMIN'. The main content area is titled 'Employee Details' with the subtitle 'Oracle employee record workspace'. Below this, there are three input fields labeled 'EMPLOYEE ID', 'NAME', and 'GENDER'.

User Dashboard

Advanced Employee Management System

Employee Management System user1 | USER

Employee Details
Oracle employee record workspace

EMPLOYEE ID:
NAME:
GENDER:
DEPARTMENT:
SALARY:
CONTACT:
Min Salary: Max Salary:

Employee Records

5.2 CRUD Operations

The system implements all four fundamental database operations:

- **Insert:** Adds a new employee record after validation. Prevents duplicate Employee IDs and duplicate contact numbers.
- **Update:** Modifies an existing record. Intelligent update detection checks whether data was actually changed before issuing an SQL UPDATE.
- **Delete:** Removes a record after displaying a confirmation dialog to prevent accidental deletion.
- **Search / Show All:** Retrieves records based on various filters or displays all records.

Insert Operation:

Employee Details
Oracle employee record workspace

EMPLOYEE ID:
NAME:
GENDER:
DEPARTMENT:
SALARY:
CONTACT:
Min Salary: Max Salary:

Employee Records

Employee ID	Name	Gender	Department	Salary	Contact
4	Asad	Male	Civil	1700	0323423221
5	Tayyaba	Female	CS	1600	03234232211
6	Safwan	Male	SE	16001	033234232214

Update Operation:

Employee Details
Oracle employee record workspace

EMPLOYEE ID
6

NAME
Safwan

GENDER
Male

DEPARTMENT
CP

SALARY
1900

CONTACT
03121111111

Min Salary
Max Salary

Insert

Update

Delete

Smart Search

Show All

Export CSV

Export TXT

Employee Records

Employee ID	Name	Gender	Department	Salary	Contact
5	Tayyaba	Female	CS	1600	03234232211
6	Safwan	Male	CP	1900	03121111111

Delete Operation:

Employee Details
Oracle employee record workspace

EMPLOYEE ID
6

NAME
Safwan

GENDER
Male

DEPARTMENT
CP

SALARY
1900

CONTACT
03121111111

Min Salary
Max Salary

Insert

Update

Delete

Smart Search

Show All

Export CSV

Export TXT

Employee Records

Employee ID	Name	Gender	Department	Salary	Contact
4	Asad	Male	Civil	1700	03234232211
5	Tayyaba	Female	CS	1600	03234232211

5.3 Smart Dynamic Search

A single unified search function handles multiple filter combinations:

- Employee ID search overrides all other filters (exact match).
- Name and Department support partial, case-insensitive matching using SQL LIKE with UPPER().
- Gender filter for exact match.
- Salary Range: both Min and Max entered searches within range; only Min finds salary $\geq$ Min; only Max finds salary $\leq$ Max.

Smart Search Operation

Employee Details

Oracle employee record workspace

EMPLOYEE ID 	NAME 	GENDER
DEPARTMENT SE	SALARY 	CONTACT
Min Salary		Max Salary
InsertUpdateDeleteSmart SearchShow AllExport CSVExport TXT		

Employee Records

Employee ID	Name	Gender	Department	Salary	Contact
1	Arsian fareed	Male	SE	1200	1232123
2	Khuram	Male	SE	1500	03313235733

5.4 Input Validation

All input fields are validated before any database operation:

- Employee ID: must be numeric and is required for insert/update/delete.
- Name: must contain alphabets only and is required.
- Salary: must be numeric and non-negative.
- Contact: must be numeric with a valid length.

Error messages are displayed to the user when validation fails, preventing invalid data from being written to the database.

5.5 Treeview Enhancements

- Vertical scrollbar for navigating large datasets.
- Row selection auto-fills all input fields in the form for easy update/delete.
- Auto-refresh after every Insert, Update, and Delete operation.
- Conditional row highlighting: rows are color-coded based on salary thresholds (e.g., green for high salary, yellow for low salary).

3	Maryam	Female	Nursing	1000	0323423443
4	Asad	Male	Civil	1700	0323423221

5.6 Multi-Level Sorting

Records in the Treeview can be sorted by Name, Salary, or Department in both Ascending and Descending order. Sorting works in conjunction with any active search filters.

5.7 Real-Time Live Search

As the user types in the Name or Department fields, the Treeview filters and updates results instantly without requiring the user to press a search button. This provides a smooth, responsive user experience.

5.8 Statistics Module

An analytics panel displays salary statistics calculated from the current employee data:

- Highest Salary
- Lowest Salary
- Average Salary
- Total Salary Expense
- Department-wise employee count

5.9 Export Feature

ADMIN users can export the currently displayed employee records to:

- CSV format — comma-separated values file compatible with Excel and spreadsheet tools.
- TXT format — plain text file for simple record keeping.

Export buttons are disabled for USER role accounts.

5.10 Safe Delete Confirmation

Before any deletion, a confirmation dialog box is displayed asking the user to confirm the action. This prevents accidental loss of data.

6. Role-Based Access Summary

Feature	ADMIN	USER
Insert Employee	✓ Enabled	✗ Disabled
Update Employee	✓ Enabled	✗ Disabled
Delete Employee	✓ Enabled	✗ Disabled
Search Employee	✓ Enabled	✓ Enabled
View All Records	✓ Enabled	✓ Enabled
Export CSV / TXT	✓ Enabled	✗ Disabled
Statistics Module	✓ Enabled	✗ Disabled

7. Application Flow

The application follows a structured flow from login to full interaction:

- Step 1 — Login Window opens on application start.
- Step 2 — User enters credentials; system queries USERS table with parameterized SQL.
- Step 3 — On successful authentication, the main Employee Management Dashboard opens.
- Step 4 — Buttons are enabled/disabled based on the logged-in user's role.
- Step 5 — ADMIN can perform full CRUD, search, filter, sort, export, and view statistics.
- Step 6 — USER can only search, view records, and use sorting.

[SCREENSHOT: Oracle SQL Developer showing table data and SQL script — insert screenshot here]

8. Oracle Table Creation Scripts

8.1 Employees Table & USERS Table



```
CREATE TABLE Employees (  
  EmpID NUMBER PRIMARY KEY,  
  Name VARCHAR2(50),  
  Gender VARCHAR2(10),  
  Department VARCHAR2(50),  
  Salary NUMBER,  
  Contact VARCHAR2(20) )  
CREATE TABLE USERS (  
  Username VARCHAR2(50) PRIMARY KEY,  
  Password VARCHAR2(50) NOT NULL,  
  Role VARCHAR2(10) CHECK (Role IN ('ADMIN', 'USER'))  
);  
/INSERT INTO USERS VALUES ('admin', 'admin123', 'ADMIN');  
INSERT INTO USERS VALUES ('user1', 'user123', 'USER');  
COMMIT;  
SELECT * FROM Employees;
```

8.3 Sample Data

EMPID	NAME	GENDER	DEPARTMENT	SALARY	CONTACT
1	Arslan fareed	Male	SE	1200	1232123
2	Khuram	Male	SE	1500	03313235733
3	Maryam	Female	Nursing	1000	0323423443
4	Asad	Male	Civil	1700	0323423221
5	Tayyaba	Female	CS	1600	03234232211

9. Challenges Faced & Solutions

Challenge	Solution
Managing role-based button states after login	Used tkinter widget config(state=NORMAL/DISABLED) based on role variable returned from login
Preventing SQL injection	Used oracledb parameterized queries with :1 placeholders for all user inputs
Smart search with multiple optional filters	Built a single dynamic SQL query using conditional WHERE clause construction
Detecting unchanged data before update	Compared current form values against selected row values before executing UPDATE
Real-time search performance	Bound trace callbacks to StringVar for Name/Department fields and debounced queries

10. Conclusion

The Advanced Employee Management System successfully demonstrates the practical integration of Python Tkinter with Oracle Database. All required features from the assignment specification have been implemented, including a secure login system with role-based access, full CRUD operations using parameterized queries, smart search and filtering, salary statistics, export functionality, and an enhanced user interface with real-time feedback.

The project satisfies CLO-1 through hands-on implementation of database connectivity and GUI design, CLO-2 through original dynamic query logic and intelligent features like update detection and real-time search, and CLO-3 through individual effort and a clear understanding of every implemented component.
